Generating Synthetic Aneurysm CTA Data with Masks Using Generative Models

Eight methods. For each: the paper, the maths, and what we do with our dataset.

Dataset	Intracranial aneurysm CTA (Bo et al., *Patterns* 2021)
Data now	99 scan + mask pairs (Tr0001–Tr0099)
Data planned	≈1,338 scan + mask pairs (full internal set)
Prepared for	Charitha
Date	November 2026

Figures are redrawn by us from the cited papers.
"(paper)" = number from the paper. "(estimate)" = our rough guess.

Contents

How to read this document **iii**

1 Our dataset in short **1**

1.1 What we have . . . 1

1.2 Why this is hard . . . 1

1.3 Two ways to make synthetic data . . . 1

1.4 Symbols used . . . 2

2 Common steps for all methods **3**

Step 1 – Split the data first . . . 3

Step 2 – Make all scans look the same . . . 3

Step 3 – Vessel map and bone map . . . 4

Step 4 – Cut patches . . . 4

Step 5 – Library of aneurysm shapes . . . 4

Step 6 – Where to put a new aneurysm . . . 4

Step 7 – Put the patch back into the scan . . . 4

3 Method 1 – LeFusion **5**

3.1 Part A – The paper . . . 5

3.2 Part B – Our dataset . . . 7

4 Method 2 – DiffTumor **9**

4.1 Part A – The paper . . . 9

4.2 Part B – Our dataset . . . 11

5 Method 3 – 3D conditional GAN (Jin et al.) **13**

5.1 Part A – The paper . . . 13

5.2 Part B – Our dataset . . . 14

6 Method 4 – RePaint **16**

6.1 Part A – The paper . . . 16

6.2 Part B – Our dataset . . . 17

7 Method 5 – MAISI / MAISI-v2 with ControlNet and LoRA **19**

7.1 Part A – The paper . . . 19

7.2 Part B – Our dataset . . . 21

8 Method 6 – Med-DDPM **23**

8.1 Part A – The paper . . . 23
8.2 Part B – Our dataset . . . 24

9 Method 7 – MedGen3D 26
9.1 Part A – The paper . . . 26
9.2 Part B – Our dataset . . . 27

10 Method 8 – brainSPADE 29
10.1 Part A – The paper . . . 29
10.2 Part B – Our dataset . . . 30

11 Simple baseline (no AI) 32

12 One fair test for all methods 33
1. Does it look real? . . . 33
2. Is it copying? . . . 33
3. Does it help? (the real test) . . . 33

13 Dataset licence 35

14 Summary, ranking and roadmap 36
Final ranking . . . 36
How each method is trained . . . 36
Main points . . . 36
Roadmap . . . 37

References 38

How to read this document

Goal. Make new CTA scans that contain brain aneurysms, *with* an exact mask for each aneurysm. Use them to train better aneurysm segmentation models.

How each method chapter is built. Every method chapter (Chapters 3–10) stands on its own and has two parts:

Part A – The paper	**Part B – Our dataset**
A.1 Idea in simple words	B.1 What data this method needs
A.2 Basics you need (maths)	B.2 How we train it (scratch / fine-tune / LoRA)
A.3 Architecture (figure)	B.3 Step-by-step pipeline
A.4 Maths and loss	B.4 Compute
A.5 Input and output	B.5 How to check it worked
A.6 Results in the paper	B.6 Risks
	B.7 Rank for our dataset

Other chapters.

- Chapter 1: our dataset in short.
- Chapter 2: common steps that all methods share (split, preprocessing, where to place an aneurysm). Part B of each method says which of these steps it needs.
- Chapter 11: a simple non-AI baseline to compare against.
- Chapter 12: one fair test for all methods.
- Chapter 13: dataset licence.
- Chapter 14: summary, ranking and roadmap.

The eight methods, in our ranked order.

Rank	**Method**	**Type**	**Works on**
1	LeFusion	diffusion, fills a mask	small 3D patch
2	DiffTumor	latent diffusion, fills a mask	small 3D patch
3	3D cGAN (Jin et al.)	GAN, fills a mask	small 3D patch
4	RePaint	diffusion, fills a mask (no extra training)	small 3D patch
5	MAISI / MAISI-v2 + ControlNet	big pretrained model	large 3D region
6	Med-DDPM	diffusion, label map → image	3D region
7	MedGen3D	diffusion, makes mask *and* image	3D region (2.5D)
8	brainSPADE	label generator + GAN	2D slices

Two data cases.

- **Case A (now):** 99 scans, about 117 aneurysms. Good for testing code only.
- **Case B (planned):** about 1,338 scans, about 1,489 aneurysms. The ranking assumes this case.

Chapter 1

Our dataset in short

1.1 What we have

Head CT angiography (CTA) from Bo et al. [1]. In CTA, dye is in the blood, so arteries look bright.

Part	Content
Scans	99 now (≈1,338 planned); $512 \times 512 \times N$ voxels; values in HU, -1024 to 3071
Internal masks	1,338 (1,186 train, 152 test of which 50 have no aneurysm); 1 = aneurysm
External masks	138 from two other hospitals + CSV (age, sex, rupture, number of aneurysms)
Voxel size	0.32–0.70 mm in-plane; 0.4–1.0 mm between slices (mostly 0.4 mm)
Aneurysm size	median ≈74 mm^3; about 5 mm across (most between 3 and 11 mm)

1.2 Why this is hard

- **Very small target.** A 5 mm aneurysm is only about 11 voxels across.
- **It must sit on a vessel.** A fake aneurysm floating in brain tissue is useless.
- **No normal scans for training.** All training cases have aneurysms.
- **Big 3D files.** One scan can take 500 MB in memory, so most methods work on small boxes (patches).

1.3 Two ways to make synthetic data

1. **Fill a mask in a real scan (inpainting).** Pick a spot on a vessel, give a mask shape, let the model draw an aneurysm inside. The rest of the scan stays real. *(Methods 1–4.)*
2. **Make a whole region.** Give a label map (vessels + aneurysm) and generate a full region. *(Methods 5–8.)*

Way 1 needs less data and compute, and keeps most of the scan real. That is why methods 1–4 rank higher.

1.4 Symbols used

Symbol	Meaning
$\boldsymbol{x}$	a CT patch (normalised HU)
$\boldsymbol{m}$	aneurysm mask (1 = aneurysm)
$\boldsymbol{v}$	vessel map
$\mathcal{E},\ \mathcal{D}$	encoder, decoder
$\boldsymbol{z}$	latent code (compressed image)
$t,\ T$	diffusion step, number of steps
$\beta_t,\ \alpha_t = 1 - \beta_t,\ \bar{\alpha}_t = \prod_{s \le t} \alpha_s$	noise schedule
$\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$	random Gaussian noise
$\boldsymbol{\epsilon}_\theta$	network that predicts noise
$\odot$	multiply element by element
$G,\ D$	GAN generator, discriminator

Chapter 2

Common steps for all methods

All eight methods need the same groundwork. We describe it once here. Each method's Part B lists which of these steps it uses.

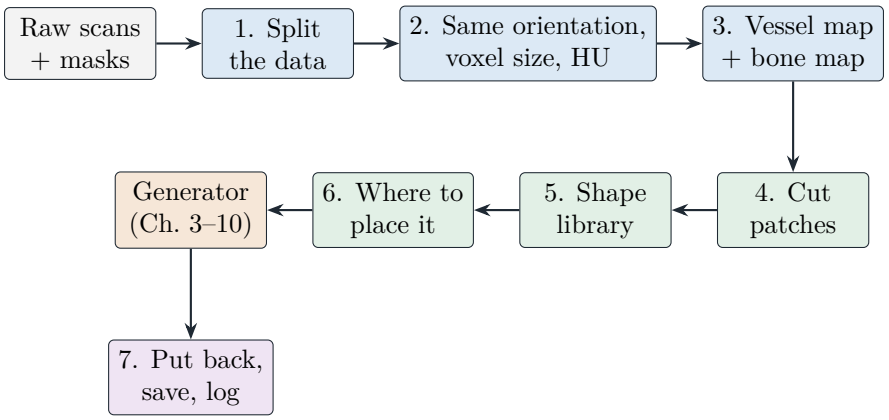


Figure 2.1: Common steps. Only the green "Generator" box changes between methods.

Step 1 – Split the data first

Set	Cases	Used for
Train	≈1,000 `Tr`	training generators and segmenters
Validation	≈186 `Tr`	choosing settings; never seen by generators
Test	152 `Ts`	final test only
External	138 `Ext` (needs scans)	final outside test only

Watch out

Train generators on the **train** set only. If they see validation or test scans, results will look better than they really are. In Case A (99 scans) use about 80 for training and 19 for validation.

Step 2 – Make all scans look the same

1. **Same orientation** (RAS) with `nib.as_closest_canonical`. This also fixes Tr0370.
2. **Same voxel size**: 0.45 mm in all directions. Linear interpolation for scans, nearest for masks.
3. **Same value range**: keep HU between −100 and 900 and scale to $[-1, 1]$:

$$\tilde{x} = 2 \cdot \frac{\mathrm{clip}(x, -100, 900) + 100}{1000} - 1. \tag{2.1}$$

4. **Clean masks**: delete blobs under 20 voxels (e.g. in Tr0734).

Step 3 – Vessel map and bone map

$$\text{bone} = \text{grow}_{2\,\text{mm}}\Big(\text{big blobs of } (x > 700\,\text{HU})\Big), \tag{2.2}$$

$$\boldsymbol{v} = (x > 200\,\text{HU}) \text{ and not bone}. \tag{2.3}$$

Near the skull base the carotid artery touches bone and may be removed. Check this area by eye.

Step 4 – Cut patches

Patch	Size	How many (Case B)	What is in it
Aneurysm patch	64^3 ($\approx$29 mm)	$\approx$1,300	a real aneurysm in the centre
Normal vessel patch	64^3	$\approx$5,000	vessel branch, far from aneurysms
Region crop	128^3–256^3	$\approx$1,000	area around the circle of Willis

All patches together are a few GB, so they are easy to upload to a cloud GPU.

Step 5 – Library of aneurysm shapes

Masks are useful even without scans. From the 1,338 + 138 masks we keep every aneurysm shape, its size and its location. We can make more shapes by random scaling (0.7–1.4×), rotation and small bending.

Step 6 – Where to put a new aneurysm

1. **Find branch points**: thin the vessel map to centre lines; points with 3 or more neighbours are branches.
2. **Prefer common places**: real aneurysm locations give a probability map

$$p(\boldsymbol{u}) = \frac{1}{K}\sum_{k=1}^{K} \mathcal{N}(\boldsymbol{u};\ \boldsymbol{u}_k,\ \sigma^2 \boldsymbol{I}), \tag{2.4}$$

   where $\boldsymbol{u}_k$ are real aneurysm centres in a normalised head frame.
3. **Pick a size** from the real size distribution (more small ones if we want).
4. **Attach the shape** to the vessel wall. Accept it only if it touches the vessel a little and does not touch bone:

$$0.05 \le \frac{|\boldsymbol{m} \cap \boldsymbol{v}|}{|\boldsymbol{m}|} \le 0.35, \qquad |\boldsymbol{m} \cap \text{bone}| = 0. \tag{2.5}$$

Step 7 – Put the patch back into the scan

Use a soft edge so no seam is visible:

$$\boldsymbol{x}^{\text{new}} = w \odot \hat{\boldsymbol{x}} + (1 - w) \odot \boldsymbol{x}, \qquad w = \text{blur}(\text{grow}_3(\boldsymbol{m})). \tag{2.6}$$

Save the new scan and mask with the *original* header. Log the source case, place, size, method and random seed. Name files `Syn_...` so they are never mixed with real data.

Chapter 3

Method 1 – LeFusion

3.1 Part A – The paper

A.1 Idea

In one line

A 3D diffusion model learns to draw *only the lesion* inside a mask. At every step, the area outside the mask is copied from a real scan. A histogram input controls how bright the lesion is. A second model (DiffMask) makes new mask shapes.

Paper. H. Zhang, Y. Liu, J. Yang, *et al.*, "LeFusion: Controllable Pathology Synthesis via Lesion-Focused Diffusion Models", ICLR 2025 [2]. Code: `https://github.com/HINTLab/LeFusion`. Used in the paper for lung nodules in CT and heart lesions in MRI.

A.2 Basics you need: diffusion models

A diffusion model learns to remove noise.

Adding noise. Take a clean image $\boldsymbol{x}_0$ and add noise in T small steps. We can jump to any step t directly:

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}). \tag{3.1}$$

When $t = T$, $\boldsymbol{x}_T$ is pure noise.

Training. A network $\boldsymbol{\epsilon}_\theta$ looks at $\boldsymbol{x}_t$ and guesses the noise $\boldsymbol{\epsilon}$ [13]:

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}_0, \boldsymbol{\epsilon}, t}\Big[\, \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)\|_2^2 \,\Big]. \tag{3.2}$$

Generating. Start from noise and remove a little noise at each step:

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)\Big) + \sigma_t\,\boldsymbol{n}, \qquad \boldsymbol{n} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}). \tag{3.3}$$

Filling a mask (inpainting). Keep the known part from the real image and generate only inside the mask $\boldsymbol{m}$:

$$\boldsymbol{x}_{t-1} = \boldsymbol{m} \odot \boldsymbol{x}_{t-1}^{\text{generated}} + (1-\boldsymbol{m}) \odot \boldsymbol{x}_{t-1}^{\text{real, noised}}. \tag{3.4}$$

A.3 Architecture

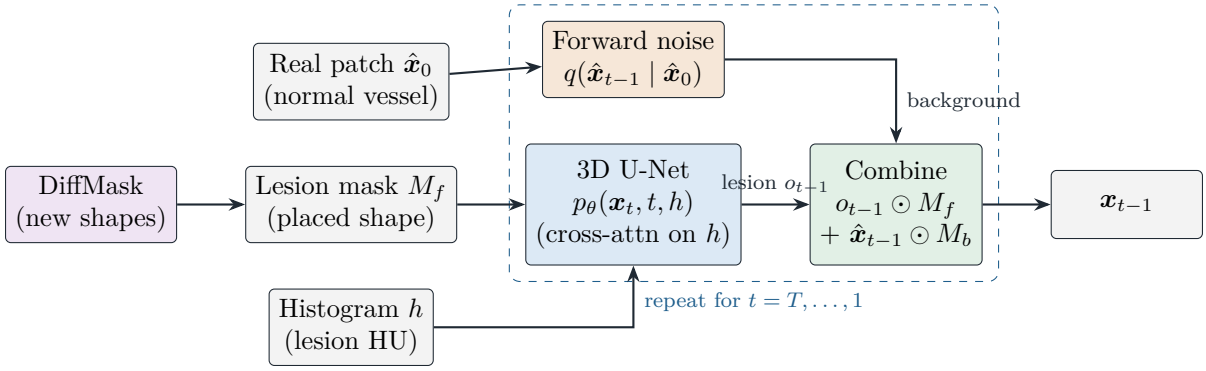


Figure 3.1: LeFusion (redrawn). The U-Net only generates the lesion; the background is a noised copy of the real patch at each step.

A.4 Maths and loss

Generation step (paper Eq. 1). Same as Eq. (3.4), with lesion mask M_f and background mask $M_b = 1 - M_f$:

$$\boldsymbol{x}_{t-1} = o_{t-1} \odot M_f + \hat{\boldsymbol{x}}_{t-1} \odot M_b, \qquad o_{t-1} \sim p_\theta(\boldsymbol{x}_t, t), \qquad \hat{\boldsymbol{x}}_{t-1} \sim q(\hat{\boldsymbol{x}}_0, t). \tag{3.5}$$

Lesion-focused loss (paper Eq. 2). The loss counts only voxels inside the lesion:

$$\mathcal{L}_{\text{LeF}} = \mathbb{E}_{\hat{\boldsymbol{x}}_0, \boldsymbol{\epsilon}, t}\Big[M_f \odot \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\hat{\boldsymbol{x}}_t, t)\|_2 \Big]. \tag{3.6}$$

Why: most voxels in a patch are background, and the background is copied anyway. So all learning goes to the lesion.

Histogram control (paper Eq. 3). A histogram h of lesion brightness is given through cross-attention:

$$o_{t-1} \sim p_\theta(\boldsymbol{x}_t, h, t). \tag{3.7}$$

Change $h \rightarrow$ the lesion becomes brighter or darker.

Several lesion types (paper Eq. 4). One mask channel per type $i = 1..n$:

$$\mathcal{L} = \mathbb{E} \sum_{i=1}^{n} \Big[M_f^{(i)} \odot \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\hat{\boldsymbol{x}}_t, t)\|_2^{(i)} \Big]. \tag{3.8}$$

DiffMask. A separate diffusion model that makes mask shapes. It gets a "control sphere" (where and how big) and returns a realistic shape inside it.

A.5 Input and output

Training input	real lesion patch, its mask M_f, histogram h
Generation input	normal patch, a mask M_f, a histogram h
Output	patch with a new lesion inside M_f; background unchanged
Loss	$\mathcal{L}_{\text{LeF}}$ (noise error inside the lesion only)

A.6 Results in the paper

Lung nodules (LIDC), Dice: nnU-Net 78.26 $\rightarrow$ 83.44; SwinUNETR 78.38 $\rightarrow$ 83.13 when LeFusion data is added. Training: 3×A100 for about 1 day. About 30 s per sample.

3.2 Part B – Our dataset

B.1 What data this method needs

- ≈1,300 aneurysm patches (64^3) with masks – from Common Steps 1–4.
- ≈5,000 normal vessel patches – the places where new aneurysms go.
- The shape library (Step 5) – to train DiffMask. Masks without scans can be used here.

B.2 How we train it

Part	Choice
LeFusion U-Net	**train from scratch**, all layers. (Option: start from the released lung model and fine-tune all layers. Do not freeze anything: the anatomy is different.)
DiffMask	**train from scratch** on our mask shapes
LoRA	**not needed** – the model is mid-size, not a huge pretrained model

B.3 Step-by-step pipeline

1. Run Common Steps 1–5.
2. For each aneurysm patch, compute h: a 16-bin histogram of HU inside the mask.
3. Change the paper's patch size $64 \times 64 \times 32$ to 64^3 (our voxels are about the same size in all directions).
4. **Train LeFusion**: $T = 1000$ steps (cosine schedule), batch 4–8, AdamW, learning rate 10^{-4}, about 100k iterations, EMA 0.999, random flips and 90° rotations.
5. **Train DiffMask** on the shape library, with a control sphere of the wanted size.
6. **Generate**: pick a normal vessel patch → choose a place (Step 6) → get a shape (DiffMask or library) → pick h from real histograms → run LeFusion.
7. Put the patch back into the full scan and save (Step 7).

B.4 Compute

One GPU with 24–40 GB. Training about 1–3 days (estimate). Generation about 30 s per aneurysm (paper).

B.5 How to check it worked

- Look at 30–50 results with the visualisation notebook. The aneurysm should grow from the vessel, with no seam.
- Compare HU inside synthetic vs. real aneurysms (mean, spread).
- Copy check: no synthetic aneurysm should be almost identical to a training one.
- Then the full test in Chapter 12.

B.6 Risks

Watch out

The mask edge must touch a bright vessel, or the result is a "floating blob". Placement (Step 6) matters as much as the model. In Case A (117 aneurysms) the model may copy training examples; use Case A only to test the code.

B.7 Rank for our dataset

Rank 1 of 8 – best fit

Small-lesion detail	★★★★★ (full resolution)
Fits our data size	★★★★☆
Compute (5 = cheap)	★★★★☆
Code available	★★★★☆
Mask is exact	★★★★★
Effort (5 = easy)	★★★☆☆

Built for exactly this job: small lesions inside real CT, at full resolution, with brightness control and new shapes.

Chapter 4

Method 2 – DiffTumor

4.1 Part A – The paper

A.1 Idea

In one line

First train a 3D autoencoder that compresses CT. Then train a diffusion model on the compressed version that fills a masked area with a lesion, using the healthy area around it as a guide. Decode and paste the lesion into a healthy scan.

Paper. Q. Chen, X. Chen, H. Song, *et al.*, "Towards Generalizable Tumor Synthesis", CVPR 2024 [3]. Code and trained models: `https://github.com/MrGiovanni/DiffTumor`. Key message: small, early lesions look alike in different organs, so the model generalises.

A.2 Basics you need: autoencoders and latent diffusion

Autoencoder. An encoder $\mathcal{E}$ makes a small code $\boldsymbol{z} = \mathcal{E}(\boldsymbol{x})$, f times smaller per axis. A decoder makes the image back: $\hat{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z})$. Training makes $\hat{\boldsymbol{x}}$ close to $\boldsymbol{x}$.

VQ (vector quantisation). Each code vector is replaced by the nearest entry $\boldsymbol{c}_z$ in a learned "codebook" [17]. Loss for this (sg = stop gradient):

$$\mathcal{L}_{\text{VQ}} = \|\text{sg}[\mathcal{E}(\boldsymbol{x})] - \boldsymbol{c}_z\|_2^2 + \gamma \, \|\text{sg}[\boldsymbol{c}_z] - \mathcal{E}(\boldsymbol{x})\|_2^2 \,. \tag{4.1}$$

Latent diffusion. Run the diffusion model (Eq. (3.1)–(3.3) in Chapter 3) on $\boldsymbol{z}$ instead of $\boldsymbol{x}$ [16]. It is faster and uses less memory.

Catch for us. With $f = 4$, a 5 mm aneurysm (11 voxels) becomes about 3 code voxels. Fine detail can be lost.

A.3 Architecture

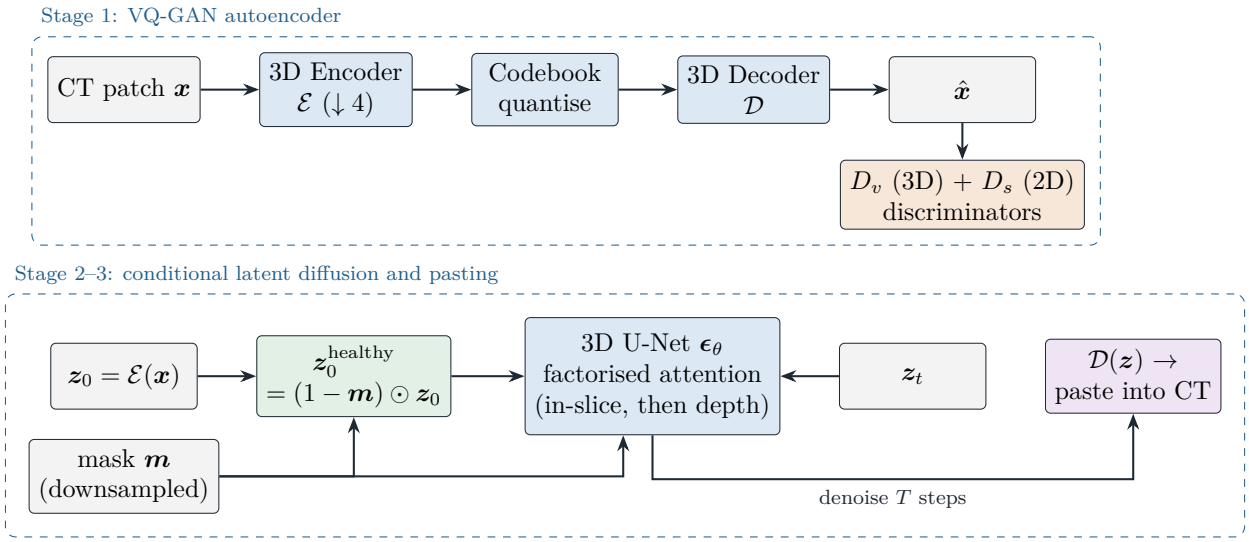


Figure 4.1: DiffTumor (redrawn). Stage 1 learns a compact CT code. Stage 2 fills the masked latent with a lesion, conditioned on the healthy latent and the mask.

A.4 Maths and loss

Stage 1 – autoencoder (paper Eq. 2). A 3D VQ-GAN, compression 4 per axis, codebook of 16,384 entries:

$$\min_{\mathcal{E},\mathcal{D},C} \Big(\mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{VQ}} + \mathcal{L}_{\text{match}} + \mathcal{L}_{\text{perc}} \Big) + \min_{\mathcal{E},\mathcal{D},C} \max_{D_s,D_v} \mathcal{L}_{\text{disc}}, \tag{4.2}$$

$$\mathcal{L}_{\text{recon}} = \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_1, \qquad \mathcal{L}_{\text{disc}} = \log D(\boldsymbol{x}) + \log\big(1 - D(\hat{\boldsymbol{x}})\big), \tag{4.3}$$

$$\mathcal{L}_{\text{match}} = \textstyle\sum_i \left\| D^{(i)}(\hat{\boldsymbol{x}}) - D^{(i)}(\boldsymbol{x}) \right\|_1, \qquad \mathcal{L}_{\text{perc}} = \text{perceptual loss on 2D slices.} \tag{4.4}$$

D_v judges 3D volumes, D_s judges 2D slices.

Stage 2 – diffusion (paper Eq. 6). The model sees the noisy code $\boldsymbol{z}_t$, the healthy code $\boldsymbol{z}_0^{\text{healthy}} = (1-\boldsymbol{m}) \odot \boldsymbol{z}_0$, and the mask $\boldsymbol{m}$:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{\boldsymbol{z}_0,\boldsymbol{\epsilon},t}\Big[\Big\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t,\ \boldsymbol{z}_0^{\text{healthy}},\ \boldsymbol{m},\ t) \Big\|_2^2 \Big]. \tag{4.5}$$

The U-Net uses split attention: first inside each slice, then across slices. Generation uses very few steps (default $T = 4$, $\approx$0.2 s per lesion).

Stage 3. Decode and paste into a healthy scan. Paper masks are ellipsoids.

A.5 Input and output

Autoencoder	CT patch in $\rightarrow$ same patch out
Diffusion input	noisy code, healthy code, mask, step t
Output	CT patch with a new lesion; mask = $\boldsymbol{m}$
Losses	AE: L_1 + VQ + perceptual + adversarial + feature match. Diffusion: $\mathcal{L}_{\text{diff}}$

A.6 Results in the paper

Dice (%), real $\rightarrow$ real + DiffTumor, U-Net: liver 62.3 $\rightarrow$ 70.9; pancreas 56.0 $\rightarrow$ 64.8; kidney 75.1 $\rightarrow$ 84.2. Autoencoder trained on 9,262 CT scans (1 mm, 96^3 crops, HU $[-175, 250]$).

4.2 Part B – Our dataset

B.1 What data this method needs

- Many patches (aneurysm + normal vessel, ≈6,000+) for the autoencoder.
- ≈1,300 aneurysm patches with masks for the diffusion model.
- Shape library (Step 5) for masks at generation time (better than ellipsoids).

B.2 How we train it

Part	Quick option	Better option (recommended)
Autoencoder	start from released CT model; **fine-tune all layers** with our HU range	**train from scratch** with compression 2 (not 4), so small aneurysms keep detail
Diffusion U-Net	**train from scratch**	**train from scratch**
LoRA	not useful (models are mid-size, data looks very different)	not useful

B.3 Step-by-step pipeline

1. Run Common Steps 1–5.
2. **Train the autoencoder** on all patches (HU $[-100, 900] \to [-1, 1]$).
3. **Test the autoencoder on aneurysms**: encode and decode 50 aneurysm patches; the aneurysm (threshold at vessel HU) should keep Dice ≥ 0.85. If not, the autoencoder is too lossy – use compression 2.
4. **Encode** all aneurysm patches. Shrink masks with max-pooling so small masks do not disappear.
5. **Train the diffusion model** with $\mathcal{L}_{\text{diff}}$, $T = 1000$. Generate with 50 DDIM steps (safer than 4 for a new kind of data).
6. **Generate**: normal vessel patch + placed mask (Step 6, shape from library) → new aneurysm.
7. Put back and save (Step 7).

B.4 Compute

Paper: autoencoder 4×A100 for about a week; diffusion 1×A100 for about a day. Ours (estimate): autoencoder 3–7 days and diffusion about 1 day on one 40 GB GPU. Generation is under a second.

B.5 How to check it worked

- Autoencoder test in step 3 (most important).
- Background HU before vs. after: decoding may shift it slightly.
- Visual check and copy check, then Chapter 12.

B.6 Risks

Watch out

Compression is the main risk: with compression 4, a 3 mm aneurysm is 1–2 code voxels. Also, decoding can change the background a little, so always paste with the soft mask.

B.7 Rank for our dataset

Rank 2 of 8

Small-lesion detail	★★★☆☆ (better with compression 2)
Fits our data size	★★★★☆
Compute (5 = cheap)	★★★☆☆
Code available	★★★★★
Mask is exact	★★★★☆
Effort (5 = easy)	★★★☆☆

Most mature code. Second only because compression can blur tiny aneurysms. First backup if LeFusion has problems.

Chapter 5

Method 3 – 3D conditional GAN (Jin et al.)

5.1 Part A – The paper

A.1 Idea

In one line

Erase the centre of a 3D CT box. A generator fills it back in. A discriminator judges if the result looks real. If the erased part held a lesion, the generator learns to draw lesions that blend in.

Paper. D. Jin, Z. Xu, Y. Tang, *et al.*, "CT-Realistic Lung Nodule Simulation from 3D Conditional Generative Adversarial Networks for Robust Lung Segmentation", MICCAI 2018 [4]. Code: not released.

A.2 Basics you need: GANs

A GAN has two networks [20]. The **generator** G makes images. The **discriminator** D says "real" or "fake". They compete: G tries to fool D. In a *conditional* GAN [21], both also see an input $\boldsymbol{x}$ (here: the box with a hole):

$$\mathcal{L}_{\text{cGAN}}(G, D) = \mathbb{E}_{\boldsymbol{x},\boldsymbol{y}}\big[\log D(\boldsymbol{x}, \boldsymbol{y})\big] + \mathbb{E}_{\boldsymbol{x}}\big[\log\big(1 - D(\boldsymbol{x}, G(\boldsymbol{x}))\big)\big]. \tag{5.1}$$

GANs are very fast at generation (one pass), but harder to train than diffusion, and can produce too little variety ("mode collapse").

A.3 Architecture

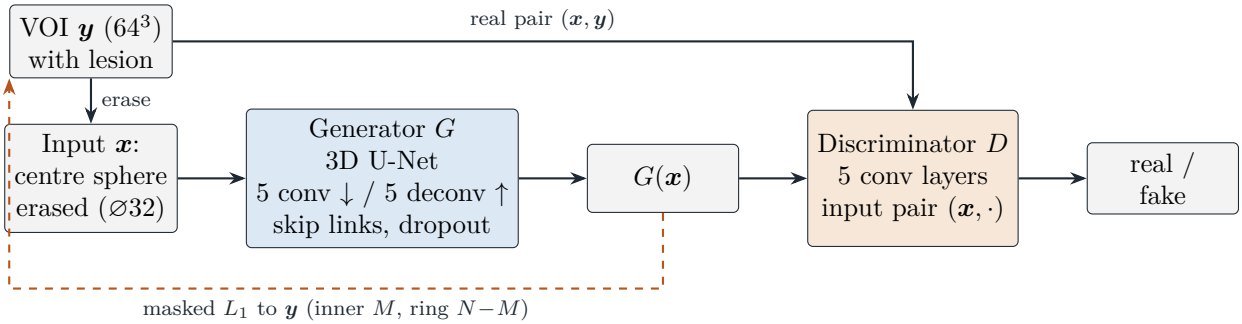


Figure 5.1: 3D cGAN inpainting (redrawn from Jin et al.). The generator fills the erased sphere; losses push the fill to match the real lesion and to look real to D.

Details. Boxes of $64 \times 64 \times 64$. The hole is a sphere of 32 voxels across. G: 3D U-Net (5 layers down, 5 up, skip links); dropout is the only randomness. D: 5 convolution layers.

A.4 Maths and loss

Adversarial loss: Eq. (5.1).

Two-zone L_1 loss. M = the hole. N = the hole grown by 3–6 voxels, so $N - M$ is a ring around it:

$$\mathcal{L}_{L_1}(G) = \mathbb{E}\Big[M \odot \|\boldsymbol{y} - G(\boldsymbol{x})\|_1\Big] + \alpha\,\mathbb{E}\Big[(N - M) \odot \|\boldsymbol{y} - G(\boldsymbol{x})\|_1\Big], \qquad \alpha = 5. \tag{5.2}$$

The ring gets 5× more weight, so the edge blends smoothly.

Total:

$$G^\star = \arg\min_G \max_D \; \mathcal{L}_{\text{cGAN}}(G, D) + \lambda\,\mathcal{L}_{L_1}(G), \qquad \lambda = 100. \tag{5.3}$$

Adam, learning rate 10^{-4}, $\beta_1 = 0.5$; 12 epochs; ≈4,300 training boxes.

A.5 Input and output

Input	64^3 box with the centre sphere erased
Output	the full 64^3 box, hole filled
Loss	cGAN loss + 100× two-zone L_1

A.6 Results in the paper

Lung segmentation Dice on scans with nodules at the lung edge: 0.964 → 0.989 after adding simulated nodules.

5.2 Part B – Our dataset

B.1 What data this method needs

≈1,300 aneurysm patches (64^3) with masks; normal vessel patches for generation; shape library.

B.2 How we train it

Train from scratch, all layers. No pretrained model exists and the model is small. LoRA is not relevant.

Two changes we make to the paper:

1. **Hole shaped like the aneurysm**: erase the mask grown by 2 voxels, not a fixed sphere.
2. **Give the mask as an extra input channel**, and add a loss that makes the bright part match the mask:

$$\mathcal{L}_{\text{mask}} = \text{BCE}\Big(\sigma\big(k\,(G(\boldsymbol{x}) - \tau)\big),\ \boldsymbol{m}\Big) \;\text{ inside } N. \tag{5.4}$$

   τ = vessel brightness level, k = sharpness. This makes the drawn aneurysm follow the mask, so the mask is correct.

B.3 Step-by-step pipeline

1. Run Common Steps 1–5.
2. Build training pairs: target $\boldsymbol{y}$ = aneurysm patch; input $\boldsymbol{x}$ = same patch with the grown mask erased + mask channel.
3. **Train** with $\mathcal{L}_{\text{cGAN}} + 100\,\mathcal{L}_{L_1} + \lambda_m \mathcal{L}_{\text{mask}}$ (λ_m = 1–10). Use spectral normalisation in D for stable training. Batch 8–16.

4. **Generate**: normal vessel patch + placed mask (Step 6) → erase → G.
5. Put back and save (Step 7).

B.4 Compute

One 16–24 GB GPU. Training under 1 day (estimate). Generation takes milliseconds.

B.5 How to check it worked

- Variety: generate 10 aneurysms for the *same* mask. If all look the same, variety is low.
- Check that bright voxels match the mask (Dice of thresholded output vs. mask).
- Visual check, then Chapter 12.

B.6 Risks

Watch out

GAN training can be unstable, and dropout is the only randomness, so results can look alike. Variety then comes mostly from the shape library and placement.

B.7 Rank for our dataset

Rank 3 of 8 – fast baseline

Small-lesion detail	★★★★☆	
Fits our data size	★★★★☆	
Compute (5 = cheap)	★★★★★	
Code available	★★☆☆☆	(simple to write)
Mask is exact	★★★☆☆	(needs our changes)
Effort (5 = easy)	★★★★☆	

Cheap and quick. A good first generative baseline to compare the diffusion methods against.

Chapter 6

Method 4 – RePaint

6.1 Part A – The paper

A.1 Idea

In one line

Train a normal diffusion model (no mask input). To fill a hole: at each step keep the real part, let the model draw only inside the hole, and go back and forth a few times so the two parts blend.

Paper. A. Lugmayr, M. Danelljan, A. Romero, *et al.*, "RePaint: Inpainting using Denoising Diffusion Probabilistic Models", CVPR 2022 [5]. Code (2D photos): `https://github.com/andreas128/RePaint`.

A.2 Basics you need

A standard diffusion model, trained with the plain loss of Eq. (3.2) (Chapter 3), *without* any condition. RePaint does not change training at all; it only changes how we generate.

A.3 Architecture

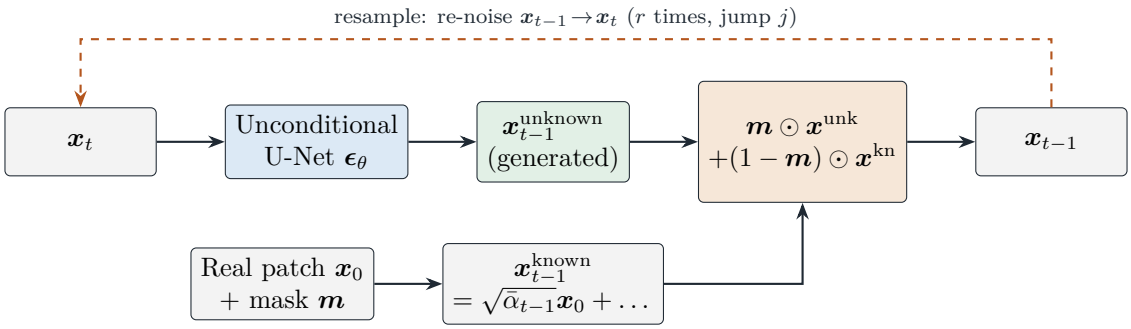


Figure 6.1: RePaint (redrawn). Known region from the real image, unknown region from the model; re-noising lets both parts agree.

A.4 Maths

With $\boldsymbol{m} = 1$ in the hole:

$$\boldsymbol{x}_{t-1}^{\text{known}} \sim \mathcal{N}\big(\sqrt{\bar{\alpha}_{t-1}}\,\boldsymbol{x}_0,\ (1-\bar{\alpha}_{t-1})\boldsymbol{I}\big) \qquad \text{(real image, noised)} \tag{6.1}$$

$$\boldsymbol{x}_{t-1}^{\text{unknown}} \sim \mathcal{N}\big(\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t),\ \Sigma_\theta(\boldsymbol{x}_t, t)\big) \qquad \text{(model's guess)} \tag{6.2}$$

$$\boldsymbol{x}_{t-1} = \boldsymbol{m} \odot \boldsymbol{x}_{t-1}^{\text{unknown}} + (1-\boldsymbol{m}) \odot \boldsymbol{x}_{t-1}^{\text{known}}. \tag{6.3}$$

Going back (resampling). The two parts may not match at the edge. So add one step of noise back,

$$\boldsymbol{x}_t \sim \mathcal{N}\big(\sqrt{1-\beta_t}\,\boldsymbol{x}_{t-1},\ \beta_t \boldsymbol{I}\big), \tag{6.4}$$

and denoise again. Repeat r times, in jumps of j steps. Paper: $T = 250$, $j = 10$, $r = 10$. This makes generation about 10× slower.

A.5 Input and output

Training input	images only (no masks)
Training loss	plain diffusion loss
Generation input	real image + hole mask
Output	image with new content in the hole

A.6 Results in the paper

Strong results for 2D photo inpainting with many hole shapes. Not tested on medical lesions in the paper.

6.2 Part B – Our dataset

B.1 What data this method needs

≈1,300 aneurysm patches (64^3) for training (masks not needed for training); normal vessel patches and the shape library for generation.

B.2 How we train it

Train a 3D diffusion model from scratch, all layers. No 3D head-CTA diffusion model exists to reuse. LoRA is not relevant. Filling the hole needs no extra training.

B.3 Step-by-step pipeline

1. Run Common Steps 1–5.
2. **Train** a 3D diffusion model on aneurysm-centred patches only ($T = 1000$, cosine). Training only on aneurysm patches teaches the model to "expect" a bright blob near the centre.
3. **Generate** with RePaint ($T = 250$, $j = 10$, $r = 10$) on a normal vessel patch, hole = placed mask grown by 1–2 voxels.
4. (Optional) push the hole to be bright with a small guidance step:

$$\boldsymbol{x}_{t-1}^{\text{unk}} \leftarrow \boldsymbol{x}_{t-1}^{\text{unk}} - \eta \nabla_{\boldsymbol{x}} \big(\text{mean of } \hat{\boldsymbol{x}}_0 \text{ inside } \boldsymbol{m} - \mu_a\big)^2. \tag{6.5}$$

5. **Fix the mask**: the model may not fill the whole hole with aneurysm, so keep only bright voxels:

$$\boldsymbol{m}^{\text{final}} = \boldsymbol{m} \wedge (\hat{\boldsymbol{x}} > \tau_v). \tag{6.6}$$

   Throw away results where $\boldsymbol{m}^{\text{final}}$ is under 70% of $\boldsymbol{m}$.
6. Put back and save (Step 7).

B.4 Compute

One 24 GB GPU; training 1–3 days (estimate). Generation about 10× slower than normal diffusion.

B.5 How to check it worked

Count how many results are thrown away in step 5 (high = poor control). Then visual check and Chapter 12.

B.6 Risks

Watch out

The model is never told "draw an aneurysm here". It may fill the hole with brain or plain vessel. This is the weakest control of the four mask-filling methods.

B.7 Rank for our dataset

Rank 4 of 8

Small-lesion detail	★★★★☆
Fits our data size	★★★☆☆
Compute (5 = cheap)	★★★☆☆ (slow generation)
Code available	★★★☆☆ (2D only; port to 3D)
Mask is exact	★★☆☆☆
Effort (5 = easy)	★★★★☆

Simple and well known. Useful as a diffusion baseline to show how much LeFusion's lesion-focused training helps.

Chapter 7

Method 5 – MAISI / MAISI-v2 with ControlNet and LoRA

7.1 Part A – The paper

A.1 Idea

In one line

Use NVIDIA's large, already-trained 3D CT generator. Keep it frozen. Train a small add-on (ControlNet) so a label map (vessels + aneurysm) controls what it draws. Optionally adapt the big model to head CTA with LoRA.

Papers. P. Guo, C. Zhao, D. Yang, *et al.*, "MAISI: Medical AI for Synthetic Imaging", WACV 2025 [6]. C. Zhao, P. Guo, D. Yang, *et al.*, "MAISI-v2", arXiv 2025 [7]. Code (Apache-2.0): `https://github.com/NVIDIA-Medtech/NV-Generate-CTMR`. Model weights: NVIDIA Open Model licence.

A.2 Basics you need

Latent diffusion: see Chapter 4, A.2.

Rectified flow (used by MAISI-v2) [19]. Instead of guessing noise, learn a "speed" that moves noise $\boldsymbol{x}_0$ to an image $\boldsymbol{x}_1$ along a straight line:

$$\boldsymbol{x}_t = (1-t)\,\boldsymbol{x}_0 + t\,\boldsymbol{x}_1, \qquad \mathcal{L}_{\text{flow}} = \mathbb{E}\Big[\,\|\boldsymbol{v}_\theta(\boldsymbol{x}_t, t, \boldsymbol{c}) - (\boldsymbol{x}_1 - \boldsymbol{x}_0)\|_2^2\,\Big]. \tag{7.1}$$

Straight lines mean good images in about 30 steps instead of 1000.

ControlNet [22]. Copy the encoder of the frozen U-Net into a trainable copy. Feed it the label map $\boldsymbol{c}$. Join it to the frozen model with "zero convolutions" $\mathcal{Z}$ that start at zero:

$$\boldsymbol{y}_c = \mathcal{F}(\boldsymbol{x};\Theta) + \mathcal{Z}\Big(\mathcal{F}\big(\boldsymbol{x} + \mathcal{Z}(\boldsymbol{c};\Theta_{z1});\Theta_c\big);\Theta_{z2}\Big). \tag{7.2}$$

Θ is frozen; only $\Theta_c, \Theta_{z1}, \Theta_{z2}$ learn. At the start the add-on outputs zero, so nothing breaks.

LoRA [23]. Fine-tune a big frozen weight W_0 cheaply by adding a small low-rank update:

$$W = W_0 + \frac{\alpha}{r}\,BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll d, k. \tag{7.3}$$

Only A and B are trained (often under 1% of the weights). B starts at zero.

A.3 Architecture

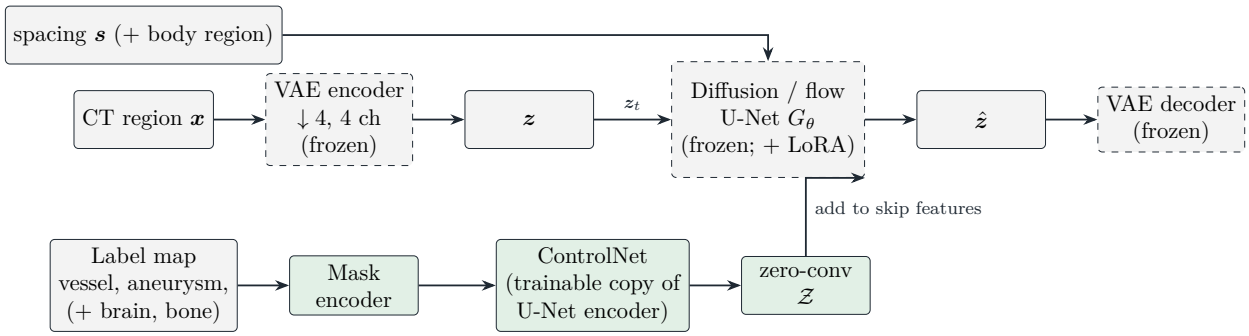


Figure 7.1: MAISI with ControlNet (redrawn). Dashed boxes are frozen. Only the ControlNet branch (and optional LoRA adapters) is trained.

A.4 Maths and loss

VAE (compression $4 \times 4 \times 4$, 4 channels):

$$\min_{\mathcal{E},\mathcal{D}} \max_{\mathcal{C}} \; \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{lpips}} + \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{adv}}, \qquad \mathcal{L}_{\text{adv}} = \log \mathcal{C}(\boldsymbol{x}) + \log\left(1 - \mathcal{C}(\mathcal{D}(\mathcal{E}(\boldsymbol{x})))\right). \tag{7.4}$$

Diffusion (v1), conditioned on body region and voxel spacing $\boldsymbol{c}_p$:

$$\mathcal{L} = \mathbb{E}\left[\, \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t, t, \boldsymbol{c}_p)\|_1 \,\right]. \tag{7.5}$$

Rectified flow (v2): Eq. (7.1).

ControlNet loss: same as the base model, with the label map added: $\mathbb{E}\left[\, \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{z}_t, t, \boldsymbol{c}_p, \boldsymbol{c}_f)\|_1 \,\right]$.

Region contrastive loss (v2). Makes the model really react to a small labelled region $\boldsymbol{m}$. Compare outputs with the real label map $\boldsymbol{c}_{\text{orig}}$ and a changed one $\boldsymbol{c}_{\text{pert}}$:

$$\mathcal{D}_{\text{roi}} = \left\| \left(G_\theta(\boldsymbol{x}_t, \boldsymbol{c}_{\text{orig}}) - G_\theta(\boldsymbol{x}_t, \boldsymbol{c}_{\text{pert}})\right) \odot \boldsymbol{m} \right\|_1, \qquad \mathcal{L}_{\text{roi}} = -\min(\mathcal{D}_{\text{roi}}, 2), \tag{7.6}$$

$$\mathcal{L}_{\text{bg}} = \left\| \left(G_\theta(\boldsymbol{x}_t, \boldsymbol{c}_{\text{orig}}) - G_\theta(\boldsymbol{x}_t, \boldsymbol{c}_{\text{pert}})\right) \odot (1 - \text{dilate}(\boldsymbol{m})) \right\|_1, \quad \mathcal{L}_{\text{total}} = \mathcal{L}_{\text{flow}} + \lambda(\mathcal{L}_{\text{roi}} + \mathcal{L}_{\text{bg}}). \tag{7.7}$$

In words: changing the aneurysm label *should* change the inside and *should not* change the outside. Very useful for tiny lesions.

A.5 Input and output

Input	label map, voxel spacing, output size, random noise
Output	new CT region that follows the label map
Trained parts	ControlNet (+ LoRA, + decoder if needed)
Loss	base loss + v2 contrastive terms

A.6 Results in the paper

v2 trained on ≈11,000 CT scans (64×A100, 3 weeks). ControlNet: 8×A100, about 2 days. Output up to $512 \times 512 \times 768$. Repo: ControlNet training needs ≥60 GB GPU at 512^3; generation fits 16–24 GB.

7.2 Part B – Our dataset

B.1 What data this method needs

- ≈1,000 region crops ($256 \times 256 \times 128$ at 0.45 mm, around the circle of Willis).
- A label map for each: vessel, aneurysm, bone, brain (Common Step 3, plus a brain mask from a threshold or a segmenter such as TotalSegmentator).

B.2 How we train it

Part	Default	If needed	Why
VAE encoder	frozen	frozen	keep the code space the same
VAE decoder	frozen	fine-tune all layers	if small vessels look blurry
Diffusion U-Net	frozen	**LoRA**, rank 8–16	adapt to head CTA without forgetting
ControlNet	**train**	train	learns our label map

LoRA goes on the attention layers (Q, K, V, O) and $1 \times 1 \times 1$ convolutions. Train LoRA first (no labels), then freeze it and train ControlNet.

B.3 Step-by-step pipeline

1. Run Common Steps 1–3. Cut region crops (Step 4).
2. **Test the VAE first**: encode and decode 20 crops. Check that vessels and aneurysms survive (Dice ≥ 0.85). MAISI was trained mostly on body CT at about 0.8–1.5 mm, so our 0.45 mm head CTA is new to it.
3. Put our labels into the repo's free (placeholder) label IDs.
4. (Optional) **LoRA** on head CTA crops.
5. **Train ControlNet** with the v2 contrastive loss (changed label = aneurysm removed or moved).
6. **Generate**: take a real label map, add or move an aneurysm at a branch (Step 6), generate the region, put back with a wide soft edge (Step 7).

B.4 Compute

40–80 GB GPU. LoRA 1–2 days, ControlNet 2–5 days (estimate). Generation: about 30 steps, seconds to minutes.

B.5 How to check it worked

VAE test (step 2); check the aneurysm appears exactly where labelled; visual check; Chapter 12.

B.6 Risks

Watch out

(1) Compression 4: a 3–5 mm aneurysm is 2–3 code voxels. (2) Head CTA at 0.45 mm may be outside what MAISI learned. (3) Needs big GPUs. Best use: realistic *regions*; let LeFusion draw the aneurysm itself.

B.7 Rank for our dataset

Rank 5 of 8

Small-lesion detail	★★☆☆☆
Fits our data size	★★★★☆ (pretrained)
Compute (5 = cheap)	★★☆☆☆
Code available	★★★★★
Mask is exact	★★★☆☆
Effort (5 = easy)	★★☆☆☆

The most powerful tool, but not made for 3–5 mm lesions in head CTA. Use it later, for region realism.

Chapter 8

Method 6 – Med-DDPM

8.1 Part A – The paper

A.1 Idea

In one line

Put a full label map (one channel per class) next to the noisy image. The 3D diffusion model learns to draw a whole image that follows the labels.

Paper. Z. Dorjsembe, H.-K. Pao, S. Odonchimed, *et al.*, "Conditional Diffusion Models for Semantic 3D Brain MRI Synthesis", IEEE JBHI 2024 [8]. Code: `https://github.com/mobaidoctor/med-ddpm`.

A.2 Basics you need

The diffusion model of Chapter 3 (A.2), with a *cosine* noise schedule [14]:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = \cos^2\!\Big(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\Big), \quad s = 0.008. \tag{8.1}$$

It adds noise more gently at the start, which helps fine detail.

A.3 Architecture

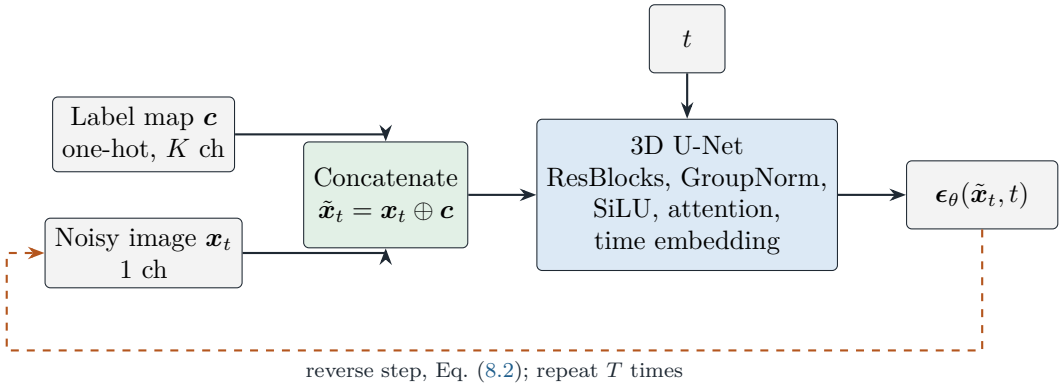


Figure 8.1: Med-DDPM (redrawn). The label map is concatenated to the noisy image at every step.

Details. 3D U-Net with ResBlocks, GroupNorm, SiLU, attention and a time embedding.

A.4 Maths and loss

Input. Stick the one-hot label map $\boldsymbol{c}$ to the noisy image: $\tilde{\boldsymbol{x}}_t = \boldsymbol{x}_t \oplus \boldsymbol{c}$.

Noise and generation as in Chapter 3, with $\boldsymbol{\epsilon}_\theta(\tilde{\boldsymbol{x}}_t, t)$:

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\Big(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\boldsymbol{\epsilon}_\theta(\tilde{\boldsymbol{x}}_t, t)\Big) + \sigma_t\,\boldsymbol{n}. \tag{8.2}$$

Loss (mean absolute error on the noise):

$$\mathcal{L}_{\text{MAE}} = \frac{1}{n}\sum_{i=1}^{n} |\epsilon_i - \epsilon_{\theta,i}(\tilde{\boldsymbol{x}}_t, t)|, \qquad n = D \cdot H \cdot W. \tag{8.3}$$

Settings: $T = 250$, cosine schedule; Adam, learning rate 10^{-5} (50k iterations) then 10^{-6} (50k); batch 1; EMA 0.995.

A.5 Input and output

Input	one-hot label map + noise
Output	full 3D image that matches the labels
Loss	$\mathcal{L}_{\text{MAE}}$

A.6 Results in the paper

1,292 training brain MRIs at 128^3. Tumour segmentation Dice: real 0.6531; synthetic only 0.6207; real + synthetic 0.6675.

8.2 Part B – Our dataset

B.1 What data this method needs

≈1,000 region crops (128^3 at 0.45 mm ≈ 58 mm cube, around the circle of Willis) with 4-class label maps: background/soft tissue, vessel, aneurysm, bone (Common Step 3).

B.2 How we train it

Train from scratch, all layers. No pretrained model of this kind exists for CTA. LoRA is not relevant.

B.3 Step-by-step pipeline

1. Run Common Steps 1–5. Cut one 128^3 crop per case.
2. Make the 4-class label map for each crop.
3. **Train** with $\mathcal{L}_{\text{MAE}}$, $T = 250$, batch 1–2, about 100k iterations. Use mixed precision and gradient checkpointing to save memory.
4. **Make new label maps**: take a real one, remove the aneurysm, add a new one at a branch (Step 6) with a shape from the library.
5. **Generate** the full crop. Use it as a training sample, or put it back with a wide soft edge.

B.4 Compute

40–80 GB GPU for 128^3; training 4–10 days (estimate); a few minutes per crop.

B.5 How to check it worked

Check that generated vessels follow the label map; compare HU of tissue, vessels and aneurysm with real crops; visual check; Chapter 12.

B.6 Risks

Watch out

The model copies the label map. Our vessel labels come from simple thresholds, so their mistakes become image mistakes. Every voxel is generated, so any odd texture can become a shortcut for the segmenter. Not possible in Case A (99 crops is too few).

B.7 Rank for our dataset

Rank 6 of 8

Small-lesion detail	★★★★☆
Fits our data size	★★☆☆☆ (from scratch)
Compute (5 = cheap)	★★☆☆☆
Code available	★★★★☆
Mask is exact	★★★★☆
Effort (5 = easy)	★★★☆☆

Simple design and full control of vessels and aneurysm. Held back by training from scratch and rough vessel labels.

Chapter 9

Method 7 – MedGen3D

9.1 Part A – The paper

A.1 Idea

In one line

First a diffusion model makes a new 3D label map, a few slices at a time. Then a video-style GAN turns the labels into an image, and three 2D diffusion models (one per direction) polish it.

Paper. K. Han, Y. Xiong, C. You, *et al.*, "MedGen3D: A Deep Generative Framework for Paired 3D Image and Mask Generation", MICCAI 2023 [9]. Code: not given in the paper. Special point: it creates *new anatomy*, not only new texture.

A.2 Basics you need: classifier-free guidance

Train one model that sometimes gets the condition and sometimes not ($\varnothing$). At generation, push toward the condition with strength s [18]:

$$\hat{\boldsymbol{\epsilon}}_\theta(X_t \mid \boldsymbol{c}) = \boldsymbol{\epsilon}_\theta(X_t \mid \varnothing) + s\,\big(\boldsymbol{\epsilon}_\theta(X_t \mid \boldsymbol{c}) - \boldsymbol{\epsilon}_\theta(X_t \mid \varnothing)\big). \tag{9.1}$$

A.3 Architecture

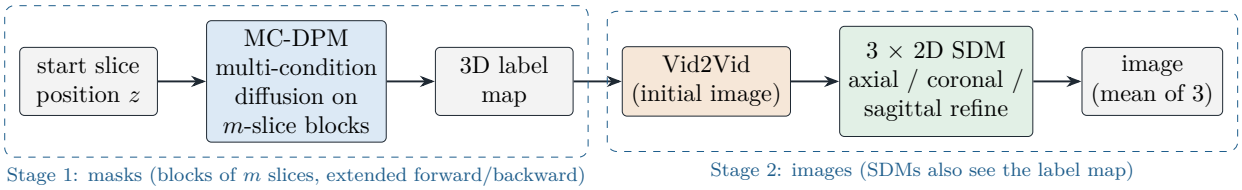


Figure 9.1: MedGen3D (redrawn). Stage 1 builds a 3D mask from slice blocks; Stage 2 paints it.

A.4 Maths and loss

Stage 1: masks (MC-DPM). Cut the label volume into blocks of $m = 6$ slices. A block is made either from nothing, or from the first n slices, or from the last n slices (picked at random in training), plus its position z/D. With X^P = slices to predict and X^C = given slices (paper Eq. 4):

$$\mathcal{L}(\theta) = \mathbb{E}_{X_0,\boldsymbol{\epsilon},p^C,z,t}\Big[\,\Big\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(X_t^P, X^C, z, t)\Big\|^2\,\Big]. \tag{9.2}$$

To make a full volume: make one block at a random place, then grow it forward and backward, $(m - n)$ slices at a time.

Stage 2: images. Vid2Vid (a video GAN) turns the slice sequence of labels into images, keeping neighbouring slices consistent. Then three 2D diffusion models (axial, coronal, sagittal) refine it with 10 small steps, and the three results are averaged.

A.5 Input and output

Stage 1	noise + position (+ known slices) → new label block
Stage 2	label map → image
Output	matched pair: image + label map

A.6 Results in the paper

Chest CT (SegTHOR), Swin-UNETR Dice: real 0.878; synthetic only 0.854; real + synthetic 0.886; pre-train on synthetic then fine-tune 0.899. Trained on small sets (30 CT, 50 MRI).

9.2 Part B – Our dataset

B.1 What data this method needs

≈1,000 region crops (≈128 slices) with label maps: vessel, aneurysm, bone, background. Masks without scans cannot train Stage 1, because the vessels around the aneurysm are missing.

B.2 How we train it

Train all five networks from scratch (MC-DPM, Vid2Vid, three 2D diffusion models). No released models. LoRA is not relevant.

B.3 Step-by-step pipeline

1. Run Common Steps 1–4; make label maps for region crops.
2. Train MC-DPM on label blocks ($m = 6$, $n = 2$–4).
3. Train Vid2Vid on label-slice → CT-slice sequences.
4. Train the three 2D diffusion models.
5. Generate new label maps, then images. Check that new aneurysms sit on vessels.

B.4 Compute

1–2 GPUs of 40 GB; 1–2 weeks for all five networks (estimate).

B.5 How to check it worked

Check new label maps first (vessels connected? aneurysms on vessels?). Look for seams between slice blocks. Then Chapter 12.

B.6 Risks

Watch out

Five networks, no official code, and a 2.5D design. Round objects that cross many slices, like aneurysms, are exactly where 2.5D methods show seams. Highest effort of all eight. Not possible in Case A.

B.7 Rank for our dataset

Rank 7 of 8

Small-lesion detail	★★★☆☆
Fits our data size	★★★☆☆
Compute (5 = cheap)	★★☆☆☆
Code available	★☆☆☆☆
Mask is exact	★★★★☆
Effort (5 = easy)	★☆☆☆☆

Nice idea (new anatomy), but too complex here. LeFusion's DiffMask already gives new shapes more simply.

Chapter 10

Method 8 – brainSPADE

10.1 Part A – The paper

A.1 Idea

In one line

Two models. A latent diffusion model invents new label maps. A SPADE GAN paints any label map as an image, in a chosen style. Together they make fully synthetic image + label pairs.

Paper. V. Fernandez, W. H. L. Pinaya, P. Borges, *et al.*, "Can Segmentation Models Be Trained with Fully Synthetically Generated Data?", SASHIMI (MICCAI workshop) 2022 [10]. Code: not released. Works on 2D slices (192×256).

A.2 Basics you need: VAE and SPADE

VAE. An autoencoder whose code is pushed to look like standard Gaussian noise, using a KL term $D_{\mathrm{KL}}(\mathcal{E}(\boldsymbol{l}) \,\|\, \mathcal{N}(\mathbf{0}, \boldsymbol{I}))$. Then new codes can be sampled.

SPADE normalisation [24]. Inside the generator, every normalisation layer is steered by the label map $\boldsymbol{l}$. For feature h at channel c and position (y, x):

$$h' = \gamma_{c,y,x}(\boldsymbol{l}) \cdot \frac{h_{c,y,x} - \mu_c}{\sigma_c} + \beta_{c,y,x}(\boldsymbol{l}). \tag{10.1}$$

γ and β come from small convolutions on $\boldsymbol{l}$, so the labels are present in every layer.

A.3 Architecture

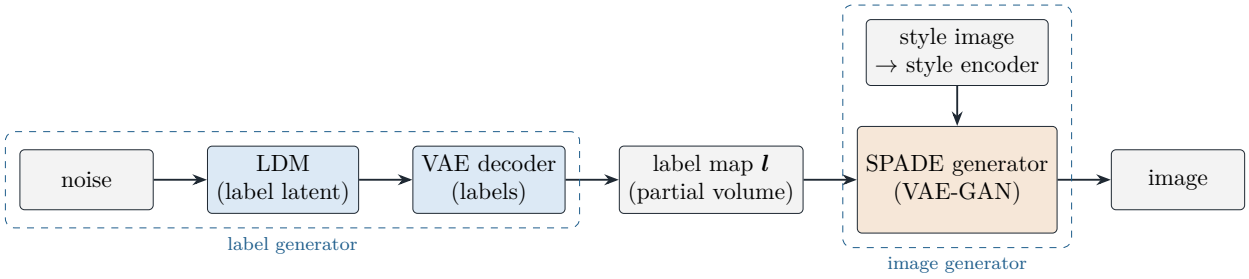


Figure 10.1: brainSPADE (redrawn). Labels are generated first, then painted by SPADE.

A.4 Maths and loss

Label generator. VAE (code $48 \times 64 \times 3$) + latent diffusion (1000 steps, L_1 loss on noise). VAE loss:

$$\mathcal{L}_{\mathrm{VAE}} = D_{\mathrm{KL}}(\mathcal{E}(\boldsymbol{l}) \| \mathcal{N}(\mathbf{0}, \boldsymbol{I})) + \mathcal{L}_{\mathrm{perc}}(\boldsymbol{l}, \hat{\boldsymbol{l}}) + \mathcal{L}_{\mathrm{adv}} + \left\| \hat{\boldsymbol{l}} - \boldsymbol{l} \right\|_1. \tag{10.2}$$

Labels are soft maps (partial volume). Separate models for healthy and tumour labels.

Image generator (paper weights):

$$\mathcal{L}_G = 1.0\, \mathcal{L}_{\mathrm{adv}}^{\mathrm{hinge}} + 0.25\, \mathcal{L}_{\mathrm{VGG}} + 0.05\, \mathcal{L}_{\mathrm{FM}} + 0.001\, \mathcal{L}_{\mathrm{KL}} + 1.0\, \mathcal{L}_{\mathrm{contrastive}} + \mathcal{L}_{\mathrm{mod,dat}}, \tag{10.3}$$

with a style-contrastive term $\mathcal{L}_{\text{contrastive}} = \text{cosim}\left(E_s(i), E_s(T(i))\right)$ and small classifiers for modality and dataset.

A.5 Input and output

Label generator	noise → 2D label map
Image generator	label map + style image → 2D image
Output	2D image + label pair

A.6 Results in the paper

Tumour Dice (2D nnU-Net): real 0.813; synthetic only 0.760; real + synthetic 0.876. Image generator trained about 2 weeks on one Quadro RTX 8000.

10.2 Part B – Our dataset

B.1 What data this method needs

Axial 2D slices from region crops, with soft label maps (vessel, aneurysm, bone, background).

B.2 How we train it

Train from scratch (label VAE, latent diffusion, SPADE GAN). No released models. LoRA is not relevant.

B.3 Step-by-step pipeline

1. Run Common Steps 1–4; cut axial slices; make soft labels by blurring the binary labels.
2. Train the label VAE + latent diffusion (a separate one for slices with aneurysm).
3. Train SPADE on (label, CT) slice pairs; style image = another slice of the same scan.
4. Generate 2D pairs. Stacking them into 3D will not be consistent.

B.4 Compute

One 24 GB GPU; 1–2 weeks (estimate). Generation is fast.

B.5 How to check it worked

Look at neighbouring generated slices: the aneurysm should not jump or change shape. Then Chapter 12 (2D version).

B.6 Risks

Watch out

An aneurysm crosses 10–25 slices. Making each slice separately breaks its 3D shape. A 3D SPADE would fix this, but that is a new research project.

B.7 Rank for our dataset

Rank 8 of 8

Small-lesion detail	★★☆☆☆
Fits our data size	★★★☆☆
Compute (5 = cheap)	★★★★☆
Code available	★☆☆☆☆
Mask is exact	★★★☆☆
Effort (5 = easy)	★★☆☆☆

Shows the idea of fully synthetic pairs well, but only useful here for 2D tests.

Chapter 11

Simple baseline (no AI)

In one line

Draw the aneurysm with simple rules: take a mask shape, fill it with vessel-like brightness, blur it like the scanner does, add matching noise.

Similar ideas helped in published work: VaMos on MRA [11] (sensitivity 75.6% → 89.0%, but false positives per scan 0.22 → 0.40) and synthetic liver tumours in CT [12].

Formula. For a placed mask $\boldsymbol{m}$, with parent-vessel brightness μ_v and spread σ_v measured nearby:

$$\boldsymbol{x}^{\text{new}} = (1 - w) \odot \boldsymbol{x} + w \odot (\mu_a + \sigma_v \cdot \text{blur}(\boldsymbol{n})), \qquad w = \text{blur}_{0.5\text{–}0.8\,\text{mm}}(\boldsymbol{m}), \quad \boldsymbol{n} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{11.1}$$

with μ_a a bit below or equal to μ_v (85–105%).

Why keep it. It takes hours to build. If a generative method cannot beat it on real test data, the extra cost is not worth it.

Chapter 12

One fair test for all methods

Every method is tested the same way, on **real** data only.

1. Does it look real?

- **Fréchet distance** between real and synthetic aneurysm patches, using features ϕ from a 3D medical network (e.g. MedicalNet [26]):

$$\mathrm{FD} = \|\boldsymbol{\mu}_r - \boldsymbol{\mu}_s\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_s - 2(\Sigma_r\Sigma_s)^{1/2}\right). \tag{12.1}$$

  Lower is better.
- **Brightness inside the aneurysm**: mean, spread and histogram vs. real.
- **Shape**: volume and roundness $\Psi = \pi^{1/3}(6V)^{2/3}/A$ vs. real.
- **Human test**: 100 real + 100 synthetic patches, mixed; can a reader tell them apart? 50% correct = cannot tell.

2. Is it copying?

For each synthetic patch, find the closest training patch:

$$d(\hat{\boldsymbol{x}}) = \min_{\boldsymbol{x}\in\text{train}} \; 1 - \cos\left(\phi(\hat{\boldsymbol{x}}), \phi(\boldsymbol{x})\right). \tag{12.2}$$

If many synthetic patches are closer to a training patch than two different real patches usually are, the model is copying.

3. Does it help? (the real test)

Train nnU-Net [27] (3d_fullres, same settings, 3 seeds) on:

Run	Training data
R	real only
R + B	real + simple baseline
R + S	real + synthetic from one method (synthetic:real = 0.5, 1, 2)
S	synthetic only (just to see)

Measure on the real test set:

- Dice on cases with aneurysms;
- found aneurysms (sensitivity), split by size: < 3, 3–5, 5–10, > 10 mm;
- false alarms per scan;
- on the 50 test scans without aneurysms: how many are correctly called "no aneurysm";
- 95% confidence intervals by bootstrap (1,000 resamples).

Watch out

Synthetic data often finds more aneurysms *and* gives more false alarms. Always report both.

Chapter 13

Dataset licence

Source. Bo ZH, Qiao H, Tian C, *et al.*, *Patterns* 2(2):100197, 2021 [1]. 1,338 internal cases from six hospitals (1,489 aneurysms) and two outside test sets.

Licence – please confirm. The *article* is CC BY-NC-ND 4.0. The *data* is reported to be for **non-commercial use**. We could not open the data website to read the exact terms. Read the terms that came with your download.

Until confirmed:

- Use the data and all synthetic data from it for non-commercial research only.
- Do not share the original scans. Treat synthetic scans the same way (they are built on real scans).
- Cite Bo et al. (2021).
- Check model licences: MAISI code Apache-2.0, weights NVIDIA Open Model licence; check LeFusion, DiffTumor and Med-DDPM licence files.
- Name synthetic files `Syn_...` and keep the log.

Chapter 14

Summary, ranking and roadmap

Final ranking (Case B, ≈1,338 scans)

Rank	Method	Lesion	Data fit	Compute	Code	Mask	Effort
1	LeFusion	5	4	4	4	5	3
2	DiffTumor	3	4	3	5	4	3
3	3D cGAN (Jin)	4	4	5	2	3	4
4	RePaint	4	3	3	3	2	4
5	MAISI-v2 + ControlNet	2	4	2	5	3	2
6	Med-DDPM	4	2	2	4	4	3
7	MedGen3D	3	3	2	1	4	1
8	brainSPADE	2	3	4	1	3	2

1 = poor, 5 = best (for Compute and Effort, 5 = cheapest/easiest). The rank is a judgement, not a sum: *lesion detail* and *exact mask* matter most.

How each method is trained

Method	Scratch / fine-tune / frozen / LoRA	GPU (estimate)
LeFusion	from scratch (option: start from lung model, fine-tune all)	24–40 GB, 1–3 days
DiffTumor	AE fine-tune all, or from scratch with compression 2; diffusion from scratch	40 GB, 4–8 days
3D cGAN	from scratch	16–24 GB, <1 day
RePaint	diffusion from scratch; filling needs no training	24 GB, 1–3 days
MAISI-v2	VAE + U-Net frozen; LoRA on U-Net; ControlNet trained	40–80 GB, 3–7 days
Med-DDPM	from scratch	40–80 GB, 4–10 days
MedGen3D	all five networks from scratch	40 GB, 1–2 weeks
brainSPADE	from scratch	24 GB, 1–2 weeks

A MacBook Air cannot train these. Use Kaggle, Colab Pro, or a rented cloud GPU. Upload patches (≈3 GB), not full scans (≈180 GB).

Main points

1. **Filling a mask in a real scan beats making whole scans** for tiny aneurysms.
2. **Full resolution beats compressed codes** for 3–5 mm lesions.
3. **Where you put the aneurysm matters as much as the model.**
4. **Masks without scans are useful** for the shape library and DiffMask.
5. **99 scans is only for testing code.** Real training needs the full download.
6. **Judge only on real test data**, and report found aneurysms *and* false alarms.

Roadmap

Phase	Work	Result
0	Download; Common Steps 1–5 (Chapter 2)	patch dataset
1	Simple baseline; nnU-Net runs R and R+B	reference numbers
2	LeFusion + DiffMask; full test	first generative result
3	3D cGAN as fast comparison; DiffTumor if LeFusion struggles	comparison
4	(Optional) MAISI-v2 + LoRA + ControlNet for regions	advanced
5	Outside test (if external scans obtained)	generalisation

Recommendation

Do the common steps and the simple baseline first. Then build **LeFusion** on 64^3 aneurysm patches, trained from scratch, with DiffMask for new shapes. Compare it with the 3D cGAN and the baseline using nnU-Net on real test data. Try MAISI-v2 only if region realism becomes the problem and big GPUs are available.

References

[1] Z.-H. Bo, H. Qiao, C. Tian, *et al.* Toward human intervention-free clinical diagnosis of intracranial aneurysm via deep neural network. *Patterns*, 2(2):100197, 2021. doi:10.1016/j.patter.2020.100197.

[2] H. Zhang, Y. Liu, J. Yang, *et al.* LeFusion: Controllable pathology synthesis via lesion-focused diffusion models. *ICLR*, 2025. arXiv:2403.14066.

[3] Q. Chen, X. Chen, H. Song, *et al.* Towards generalizable tumor synthesis. *CVPR*, 2024. arXiv:2402.19470.

[4] D. Jin, Z. Xu, Y. Tang, *et al.* CT-realistic lung nodule simulation from 3D conditional generative adversarial networks for robust lung segmentation. *MICCAI*, 2018. arXiv:1806.04051.

[5] A. Lugmayr, M. Danelljan, A. Romero, *et al.* RePaint: Inpainting using denoising diffusion probabilistic models. *CVPR*, 2022. arXiv:2201.09865.

[6] P. Guo, C. Zhao, D. Yang, *et al.* MAISI: Medical AI for synthetic imaging. *WACV*, 2025. arXiv:2409.11169.

[7] C. Zhao, P. Guo, D. Yang, *et al.* MAISI-v2: Accelerated 3D high-resolution medical image synthesis with rectified flow and region-specific contrastive loss. arXiv:2508.05772, 2025.

[8] Z. Dorjsembe, H.-K. Pao, S. Odonchimed, *et al.* Conditional diffusion models for semantic 3D brain MRI synthesis. *IEEE J. Biomed. Health Inform.*, 2024. doi:10.1109/JBHI.2024.3385504.

[9] K. Han, Y. Xiong, C. You, *et al.* MedGen3D: A deep generative framework for paired 3D image and mask generation. *MICCAI*, 2023. arXiv:2304.04106.

[10] V. Fernandez, W. H. L. Pinaya, P. Borges, *et al.* Can segmentation models be trained with fully synthetically generated data? *SASHIMI (MICCAI workshop)*, 2022. arXiv:2209.08256.

[11] VaMos authors (Université de Nantes). A vascular synthetic model for improved aneurysm segmentation and detection via deep neural networks. arXiv:2403.18734, 2024.

[12] Q. Hu, Y. Chen, J. Xiao, *et al.* Label-free liver tumor segmentation. *CVPR*, 2023. arXiv:2303.14869.

[13] J. Ho, A. Jain, P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 2020.

[14] A. Nichol, P. Dhariwal. Improved denoising diffusion probabilistic models. *ICML*, 2021.

[15] J. Song, C. Meng, S. Ermon. Denoising diffusion implicit models. *ICLR*, 2021.

[16] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer. High-resolution image synthesis with latent diffusion models. *CVPR*, 2022.

[17] P. Esser, R. Rombach, B. Ommer. Taming transformers for high-resolution image synthesis. *CVPR*, 2021.

[18] J. Ho, T. Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022.

[19] X. Liu, C. Gong, Q. Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *ICLR*, 2023.

[20] I. Goodfellow *et al.* Generative adversarial nets. *NeurIPS*, 2014.

[21] P. Isola, J.-Y. Zhu, T. Zhou, A. A. Efros. Image-to-image translation with conditional adversarial networks. *CVPR*, 2017.

[22] L. Zhang, A. Rao, M. Agrawala. Adding conditional control to text-to-image diffusion models. *ICCV*, 2023.

[23] E. J. Hu *et al.* LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

[24] T. Park, M.-Y. Liu, T.-C. Wang, J.-Y. Zhu. Semantic image synthesis with spatially-adaptive normalization. *CVPR*, 2019.

[25] A. F. Frangi, W. J. Niessen, K. L. Vincken, M. A. Viergever. Multiscale vessel enhancement filtering. *MICCAI*, 1998.

[26] S. Chen, K. Ma, Y. Zheng. Med3D: Transfer learning for 3D medical image analysis. arXiv:1904.00625, 2019.

[27] F. Isensee, P. F. Jaeger, S. A. A. Kohl, J. Petersen, K. H. Maier-Hein. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18:203–211, 2021.